



SpeedAI 论文AIGC检测报告

标题：论文10

作者：hiro

检测编号：5f018b89-cdff-48f5-b11e-4729d9394282

检测时间：2026-05-19 14:25:23

检测结果

检测模型：仿知网，与官方相差10%左右

总字数：22510

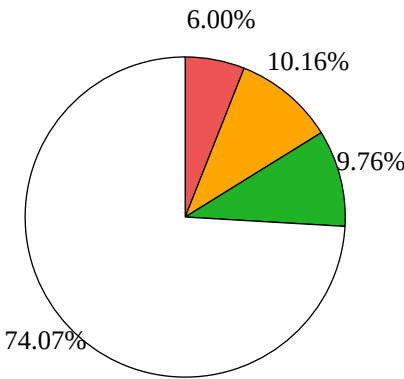
疑似AIGC风险概率：**25.93%**

高风险文本（≥90%）：1351字，占比6.00%

中风险文本（70-90%）：2288字，占比10.16%

低风险文本（50-70%）：2198字，占比9.76%

无风险文本（<50%）：16673字，占比74.07%



片段汇总列表

序号	段落内容	AI生成概率	风险等级
1	在持续数字化的背景下，传统人工点餐模式有明显的不足，比如排队、餐桌反馈、订单数据统计等。针对上述问题，本论文以点餐业务为研究对象，设计并实现了一套基于 Web 技术的订餐系统，旨在通过信息化手段提高效率。	55.85%	低风险
2	Against the backdrop of continuous digitalization, traditional manual ordering methods exhibit significant shortcomings, such as queuing delays, inefficient table feedback, and difficulties in order data statistics. To address these issues, this paper takes the food ordering business as the research object and designs and implements a Web-based ordering system aimed at improving efficiency through informatization.	98.13%	高风险
3	The system is designed around two types of users: ordinary users and administrators. The backend business services are built using Spring Boot, MyBatis, and MySQL. The Web-based administration platform is implemented with Vue.js and Element Plus to manage orders, tables, dishes, users, and recharge information. The WeChat Mini Program is designed for ordinary users and provides functions such as registration and login, seat selection, dish browsing, shopping cart ordering, order settlement, and personal profile management.	95.03%	高风险
4	The system adopts a simple and intuitive style and completes core business processes including table reservation, ordering, order placement, payment settlement, and backend maintenance. It can meet the basic requirements of food ordering scenarios and features a clear structure, complete functions, smooth operation flow, and strong scalability.	72.63%	中风险

5	MyBatis 是一种开源、轻量级的数据持久化框架，它将 SQL 编写权交由开发者控制，并通过 Mapper 接口和 XML 映射文件完成 Java 对象与数据库记录之间的转换 [2]。	69.69%	低风险
6	普通用户在微信小程序端完成账号注册或登录，系统校验身份后建立可用的登录态。	63.22%	低风险
7	注册成功时创建用户信息并赋予 USER 角色；登录成功时生成 token。	69.72%	低风险
8	普通用户查看全部餐桌状态，并查询自己当前已经占用的餐桌信息。	69.72%	低风险
9	用户已登录系统；选座时目标餐桌处于空闲状态，退桌时用户已占用餐桌。	89.86%	中风险
10	普通用户将菜品加入购物车并提交订单，系统生成订单编号和订单记录。	69.71%	低风险
11	用户已登录系统，且已选择餐桌并在购物车中添加了至少一种菜品。	89.86%	中风险
12	普通用户查看自己的历史订单，并对满足条件的订单执行余额结算。	69.71%	低风险
13	用户已登录系统；结算时订单状态为“待结算”，且用户余额充足。	91.98%	高风险
14	系统校验订单归属、订单状态和账户余额，扣减余额后将订单更新为“已完成”。	72.61%	中风险
15	普通用户查看和维护个人资料，包括姓名、手机号、性别、头像及余额展示。	69.72%	低风险
16	系统更新用户资料，并在需要时完成头像文件上传与访问路径保存。	83.98%	中风险
17	系统的主要功能模块包括用户认证与登录、餐桌管理、菜品浏览与购物车、订单处理与结算、个人中心以及后台运营管理等模块。各模块之间既相对独立，又共同服务于“选座—点餐—下一单—出餐—结算”的业务主线。	63.22%	低风险
18	用户基础服务模块主要为普通用户提供注册、登录和登录态维护功能。	51.86%	低风险
19	餐桌与点餐业务模块需要支持用户查看全部餐桌状态和当前餐桌信息，并完成占桌、退桌和占用冲突校验。点餐业务则需要支持用户浏览并下单。	93.68%	高风险
20	订单模块管理员需要记录订单编号、订单内容、订单金额和订单状态，用户支持查询个人订单并跟踪状态变化。	80.04%	中风险
21	本系统采用前后端分离的三层架构设计。表现层由 Vue 3 Web 管理端和微信小程序用户端组成；业务层由 Spring Boot 服务承担核心逻辑；数据访问层基于 MyBatis 与 MySQL 完成数据的持久化处理 [7]。	51.83%	低风险
22	用户完成选座后进入点餐页面，系统首先加载当前餐桌信息和菜品列表，并展示菜品名称、价格、图片和描述等内容。用户根据需要选择菜品数量并加入购物车，系统同步计算购物车中的菜品数量和订单总金额。当用户点击提交订单时，系统先判断用户是否已经占用餐桌以及购物车中是否存在菜品；若条件不满足，则提示用户先选座或添加菜品；若条件满足，则整理菜品名称、数量、价格和总价等信息，并向服务器发送下单请求。服务器接收请求后生成订单编号，记录下单时间，绑定当前用户信息，并将订单初始状态设置为“待出餐”。若请求处理成功，页面提示下单成功并清空购物车；若请求失败，系统提示用户重新提交订单。点餐下单流程图如图4.5。	77.77%	中风险
23	用户资料与充值管理主要用于维护个人信息和账户余额。普通用户进入个人中心页面后，系统会查询当前用户的姓名、手机号、性别、头像和余额等信息，并将结果展示在页面中。用户可以根据需要修改个人资料，若只修改文字信息，则系统直接调用用户信息更新接口保存数据；若用户需要更换头像，则先通过文件上传接口上传图片，服务器将图片保存到指定目录并返回访问地址，随后系统再把头像地址更新到用户资料中。管理员在后台可以查看用户列表和用户余额，并根据实际充值情况修改账户金额。若保存成功，系统刷新用户信息并提示操作完成；若请求失败，则提示用户重新提交。用户资料与充值管理流程图如图4.7。	63.22%	低风险

24	本系统的概念模型主要围绕管理员、用户、餐桌、菜品和订单五类核心实体展开，各实体及其属性如下：	83.98%	中风险
25	用户信息：一个用户在同一时刻最多只能占用一张餐桌，也可以处于未占桌状态。	83.99%	中风险
26	餐桌信息：一张餐桌在同一时刻最多只能被一个用户占用，也可以保持空闲状态。	80.04%	中风险
27	订单信息：一个用户可以提交多个订单，但每个订单只归属于一个用户，因此用户与订单之间构成一对多关系。	92.88%	高风险
28	订单信息：一个订单可以包含多种菜品，订单总价由所选菜品数量和价格共同决定。	77.78%	中风险
29	菜品信息：同一种菜品可以被多个用户重复点选，并出现在不同订单中。	51.87%	低风险
30	联系说明：从概念模型来看，订单与菜品之间属于多对多联系；当前系统在物理实现中将订单内容以字符串形式保存在 content 字段中，尚未拆分为独立的订单明细表。该处理方式可以简化轻量级系统的实现，但在后续扩展统计分析、退菜或单品维度结算功能时，仍有继续规范建模的空间。	80.03%	中风险
31	4) 管理员实体与用户、菜品、餐桌及订单实体之间存在“维护”和“处理”联系。	51.85%	低风险
32	联系说明：管理员主要负责用户资料维护、菜品与餐桌基础数据管理，以及订单状态处理和后台统计。	51.85%	低风险
33	登录模块主要负责普通用户注册、身份校验和登录态建立。小程序端主要通过 /auth/register 和 /auth/login 完成账号创建与登录，管理员端则保留 /login 作为兼容入口。	55.85%	低风险
34	请求体包含 username、password、name、role；其中 role 固定为 USER，用户名需保持唯一。	89.86%	中风险
35	返回统一 Result 对象，包含 code、msg、data；注册成功时 data 通常为空。	72.62%	中风险
36	请求体包含 username、password、role；普通用户 role 为 USER，管理员兼容接口可使用 /login。	55.85%	低风险
37	返回 Result 对象；data 中包含 id、username、name、avatar、role、token 等登录态信息。	85.69%	中风险
38	用于校验账号密码并签发 JWT Token，前端登录成功后将 token 保存到本地并用于后续鉴权请求。	69.72%	低风险
39	请求头携带 Authorization: Bearer <token>; 无业务请求体。	80.04%	中风险
40	返回 Result 对象；data 为当前用户信息，包含 id、username、name、avatar、sex、phone、account、role 等字段。	69.72%	低风险
41	请求头携带 Authorization: Bearer <token>; 请求体可包含 name、phone、sex、avatar 等需更新字段。	80.05%	中风险
42	返回 Result 对象；data 为更新后的最新用户信息。	82.11%	中风险
43	请求头携带 Authorization: Bearer <token>; 请求体主要包含餐桌 id，后端结合当前用户编号完成占桌。	80.04%	中风险
44	返回 Result 对象；data 为最新餐桌信息，包含 id、no、unit、free、userId、userName 等字段。	75.31%	中风险
45	用于当前用户占用指定餐桌，后端同时校验“一个用户只能占一个餐桌、一个餐桌只能被一个用户占用”的业务规则。	51.86%	低风险

46	订单结算模块贯穿下单、订单状态流转和余额扣减等关键业务，前端既需要提交购物车生成订单，也需要在待结算状态下发起结算请求。	51.85%	低风险
47	请求头携带 Authorization: Bearer <token>; 请求体包含 content、total 等订单核心数据，userId、orderNo、time 由后端自动补全。	66.58%	低风险
48	返回 Result 对象；data 为创建后的订单信息，包含订单编号、下单时间、状态和所属用户等字段。	90.98%	高风险
49	用于将前端购物车内容提交为正式订单，后端自动生成订单编号并将订单初始状态设置为“待出餐”。	80.04%	中风险
50	路径参数 id 为订单主键；请求头携带 Authorization: Bearer <token>; 请求体为空。	92.87%	高风险
51	返回 Result 对象；data 为结算后的订单信息，订单状态更新为“已完成”。	80.04%	中风险
52	用于对当前用户名下处于“待结算”状态的订单执行余额扣减和状态更新，并完成订单闭环。	80.04%	中风险
53	文件资源模块主要服务于用户头像和菜品图片管理，前端先上传文件并获得访问地址，再通过下载接口在页面中进行展示。	66.58%	低风险
54	表单参数 file，内容类型为 multipart/form-data；请求体携带待上传的头像或菜品图片文件。	72.63%	中风险
55	返回 Result 对象；data 为可直接访问的文件 URL，通常形如 /files/download/{fileName}。	96.10%	高风险
56	用于将头像或菜品图片保存到 photo 目录，并向前端返回后续可访问的资源地址。	77.78%	中风险
57	接口直接返回文件二进制流，响应头包含内容类型和文件名；文件不存在时返回 404。	83.98%	中风险
58	开发及测试平台：本系统在 Windows 10 环境下完成开发与联调，后端采用 IntelliJ IDEA 2024、JDK 21、Maven 3.9 和 MySQL 5.7，Web 管理端采用 Vue 3.5、Vite 6、Node.js 18 和 Element Plus，微信用户端使用微信开发者工具进行调试，接口测试和页面验证通过本地 9090 端口服务完成。	51.86%	低风险
59	用户占桌成功后，系统将餐桌状态由“是”更新为“否”，并写入占用用户编号；当用户退桌时，再根据当前用户编号查找已占用餐桌并释放绑定关系，从而保证餐桌状态与用户操作保持一致。关键代码如下：	93.68%	高风险
60	结算时，系统会校验订单是否存在、是否属于当前用户以及是否处于“待结算”状态。校验通过后再扣减用户余额，并将订单状态更新为“已完成”，从而实现从下单到支付完成的完整流程。关键代码如下：	75.30%	中风险
61	测试目的：验证管理员对菜品信息的新增、查询、修改和删除功能是否正常，确保菜品名称、价格、描述与图片等数据能够正确保存和回显。	75.31%	中风险
62	预期结果：新增后列表能够立即显示新菜品；修改后页面展示最新信息；删除后记录从列表中移除；整个操作过程均返回成功提示，且不会不影响其他菜品数据。	83.99%	中风险
63	测试目的：验证普通用户账号注册、登录鉴权以及登录态保存是否正常，确保用户能够顺利进入系统并访问后续业务页面。	72.62%	中风险
64	测试目的：验证“一个用户同一时间只能占用一个餐桌、一个餐桌同一时间只能被一个用户占用”的业务规则是否正确执行，并检查退桌后的状态恢复情况。	87.22%	中风险
65	测试过程：1. 用户登录小程序进入“选座”页面，查看当前餐桌列表与空闲状态。2. 选择一个空闲餐桌并提交占桌请求。3. 进入点餐页或返回选座页，检查当前餐桌信息是否正确显示。4. 使用同一用户再次尝试占用其他餐桌，观察系统提示信息。5. 执行退桌操作后刷新页面，检查原餐桌是否恢复为空闲状态。	66.58%	低风险

66	预期结果： 首次占桌成功后页面显示当前餐桌信息；再次占用其他餐桌时系统能够拦截并提示；退桌后当前餐桌信息被清空，原餐桌重新显示为空闲状态。	87.22%	中风险
67	测试目的：验证用户从点餐下单、管理员处理订单到用户完成结算的完整业务链路，并检查订单状态变更与余额扣减结果是否准确。	83.98%	中风险
68	预期结果： 订单能够成功创建；状态按照“待出餐→待结算→已完成”的顺序流转；余额扣减金额与订单总额一致；后台收入统计仅对已完成订单生效。	69.71%	低风险
69	测试结果：测试通过。 结果分析：订单模块已实现从下单到结算的业务闭环，核心状态控制、余额扣减和后台收入统计逻辑均符合系统设计要求。	51.85%	低风险
70	本文围绕订餐业务，设计并实现了一套基于 Web 的订餐系统。系统采用前后端分离架构，后端以 Spring Boot 作为核心服务框架，结合 MyBatis 和 MySQL 完成业务数据持久化；Web 管理端采用 Vue 3 与 Element Plus 构建管理员界面，微信小程序端则给普通用户点餐。	55.85%	低风险

原文内容

*中图分类号：TN384 密 级：公 开

本科毕业设计方案

学 院

信息科学与工程学院

专 业

计算机科学与技术

班 级

计科2202

学 生

黄建宏

学 号

202231222166

指导教师

张玲

二〇二六年五月十三日

济南大学毕业论文（设计）诚信承诺书

本人声明所呈交的毕业论文（设计）是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得济南大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

学生签名： 日期： 年 月 日

济南大学毕业论文（设计）使用授权书

本人完全了解济南大学有权保留本人所送交毕业论文（设计）的复印件和电子文档，可以采用影印、缩印或其他复制手段保存论文。本人电子文档的内容和纸质论文的内容相一致。除在保密期内的保密论文外，允许论文被查阅和借阅，可以公布（包括刊登）论文的全部或部分内容。的公布（包括刊登）授权济南大学教务处办理。

学生签名： 导师签名：

日期： 年 月 日 日期： 年 月 日

摘 要

在持续数字化的背景下，传统人工点餐模式有明显的不足，比如排队、餐桌反馈、订单数据统计等。针对上述问题，本论文以点餐业务为研究对象，设计并实现了一套基于 Web 技术的订餐系统，旨在通过信息化手段提高效率。

本系统围绕普通用户和管理员两类角色展开设计。后端采用 Spring Boot、MyBatis 与 MySQL 构建业务服务，Web 管理端基于 Vue 3 和 Element Plus 实现订单、餐桌、菜品、用户和充值等信息维护，微信小程序端面向普通用户提供注册登录、选座、菜品浏览、购物车下单、订单结算和个人资料维护等功能。 [1-6]。

该系统风格简洁，完成了占桌、点餐、下单、结算以及后台维护等核心业务流程，能够满足订餐场景下的基本订餐需求，具有结构清晰、功能完整、操作连贯和可扩展等特点。

关键词：订餐系统；Spring Boot；Vue 3；微信小程序；前后端分离

ABSTRACT

Against the backdrop of continuous digitalization, traditional manual ordering methods exhibit significant shortcomings, such as queuing delays, inefficient table feedback, and difficulties in order data statistics. To address these issues, this paper takes the food ordering business as the research object and designs and implements a Web-based ordering system aimed at improving efficiency through informatization.

The system is designed around two types of users: ordinary users and administrators. The backend business services are built using Spring Boot, MyBatis, and MySQL. The Web-based administration platform is implemented with Vue.js and Element Plus to manage orders, tables, dishes, users, and recharge information. The WeChat Mini Program is designed for ordinary users and provides functions such as registration and login, seat selection, dish browsing, shopping cart ordering, order settlement, and personal profile management.

The system adopts a simple and intuitive style and completes core business processes including table reservation, ordering, order placement, payment settlement, and backend maintenance. It can meet the basic requirements of food ordering scenarios and features a clear structure, complete functions, smooth operation flow, and strong scalability.

Keywords: Food Ordering System; Spring Boot; Vue 3; WeChat Mini Program; Frontend-Backend Separation

1 前言

1.1 课题研究的背景及意义

随着移动互联网、扫码下单和微信小程序等技术在生活场景中的普及，餐饮服务模式也逐渐出现了转变。对于餐厅而言，点餐、选座、出餐、结算和信息统计均属于高频且联系紧密的业务环节，如果仍主要依赖人工点餐，会导致效率低下。因此，构建能够用户点餐与后台管理的信息系统，对提升订餐运行效率具有现实意义[5][10]。

本课题切入订餐场景，采用“管理员后台管理 + 用户移动端下单”的模式展开设计。系统可以帮助普通用户较快完成选座、点餐、订单查询和结算操作；此外，系统也能够帮助管理员集中维护菜品、订单、餐桌和用户信息。因此具备很有必要的研究价值。

1.2 国内外研究现状

就国内研究情况来看，围绕餐饮服务以及微信小程序的订餐系统开发已经较为常见，相关成果大多集中在基础功能方面。政府对面高校或单位的内部订餐系统开始有了关注，但不少轻量级项目仍有改进空间，说明同类系统还需要进一步完善[10]。

国外餐饮数字化起步相对较早，在这面积累了较多实践，系统设计通常更加标准化。相比之下，国内订餐系统还需要结合微信小程序生态、网络环境和本地化管理需求进行适配，需要进一步的研究和改进。

2 相关技术介绍

2.1 Spring Boot介绍

Spring Boot 是构建在 Spring 生态基础上的快速开发框架，提供简便的方式来设置，可以减少项目初始化和环境配置所需的工作量，使开发者将更多精力集中在业务逻辑实现上 [1]。

2.2 Mybatis介绍

MyBatis 是一种开源、轻量级的数据持久化框架，它将 SQL 编写权交由开发者控制，并通过 Mapper 接口和 XML 映射文件完成 Java 对象与数据库记录之间的转换 [2]。

2.3 Vue.js介绍

Vue.js 是一种基于 JavaScript 的前端框架，具有组件化开发、响应式数据绑定和单页应用构建等能力，配合 Vue Router、Axios 与 Element Plus 形成完整的交互界面[3][4]。

2.4 微信小程序

微信小程序是一种依托微信生态运行的轻量级应用形态，具有无需下载安装、使用门槛较低和便于移动场景快速接入等特点，比较适合校园服务类系统的推广使用 [5]。

3 需求分析

3.1 需求概述

本订餐系统主要面向管理员和普通用户两类角色。管理员通过 Web 管理端完成订单、餐桌、菜品、用户、管理员和收入统计等业务维护，普通用户通过微信小程序完成注册登录、查看当前餐桌、选座退桌、浏览菜品、购物车下单、

订单查询结算以及个人资料维护等操作。就需求约束而言，系统既要覆盖完整的订餐业务链路，又要保证同一用户同一时刻只能占用一个餐桌、同一餐桌同一时刻只能被一个用户占用、订单状态按业务流程流转、余额结算结果准确、图片资源能够正常上传访问以及接口响应格式统一，从而保障系统在校园食堂场景下的可用性和管理性[10]。

3.2 用例分析

用户注册与登录的用例规约如表3. 1所示。

表3. 1用例“用户注册与登录”的用例规约

用例名称：

用户注册与登录

用例描述：

普通用户在微信小程序端完成账号注册或登录，系统校验身份后建立可用的登录态。

参与者：

普通用户

前置条件：

用户已进入登录页面；注册时账号未被占用，登录时账号已存在。

基本操作：

用户输入账号和密码，并选择注册或登录操作。

系统校验账号是否可用以及密码是否满足登录条件。

注册成功时创建用户信息并赋予 USER 角色；登录成功时生成 token。

小程序保存 token 和用户信息，并跳转到系统业务页面。

查看餐桌与当前餐桌的用例规约如表3. 2所示。

表3. 2用例“查看餐桌与当前餐桌”的用例规约

用例名称：

查看餐桌与当前餐桌

用例描述：

普通用户查看全部餐桌状态，并查询自己当前已经占用的餐桌信息。

参与者：

普通用户

前置条件：

用户已登录系统。

基本操作：

用户进入选座页面并发起当前餐桌查询请求。

系统读取当前用户的占桌记录并返回当前餐桌信息。

系统查询全部餐桌列表及其空闲或占用状态。

页面展示当前餐桌、餐桌编号、规格和可选状态。

选座与退桌的用例规约如表3. 3所示。

表3. 3用例“选座与退桌”的用例规约

用例名称：

选座与退桌

用例描述：

普通用户选择空闲餐桌进行占用，或在就餐结束后释放当前餐桌。

参与者：

普通用户

前置条件：

用户已登录系统；选座时目标餐桌处于空闲状态，退桌时用户已占用餐桌。

基本操作：

用户在餐桌列表中选择空闲餐桌并提交占桌请求。

系统校验用户是否已占用其他餐桌，以及目标餐桌是否仍为空闲。

校验通过后系统绑定用户与餐桌关系，并更新餐桌占用状态。

用户需要退桌时发起退桌请求，系统解除绑定并恢复餐桌为空闲。

浏览菜品的用例规约如表3. 4所示。

表3. 4用例“浏览菜品”的用例规约

用例名称：

浏览菜品

用例描述：

普通用户查看系统中的菜品信息，为后续加购和下单提供数据支持。

参与者：

普通用户

前置条件：

用户已登录系统。

基本操作：

用户进入点餐页面并请求菜品列表数据。

系统查询全部菜品的名称、价格、描述和图片信息。

系统返回可展示的菜品集合。

页面按列表形式展示菜品，供用户浏览和选择数量。

购物车下单的用例规约如表3.5所示。

表3.5用例“购物车下单”的用例规约

用例名称：

购物车下单

用例描述：

普通用户将菜品加入购物车并提交订单，系统生成订单编号和订单记录。

参与者：

普通用户

前置条件：

用户已登录系统，且已选择餐桌并在购物车中添加了至少一种菜品。

基本操作：

用户在菜品列表中调整数量并加入购物车。

用户确认购物车内容和总价后提交订单。

系统拼接订单内容、生成订单编号和下单时间，并绑定当前用户。

系统保存订单记录，初始状态设置为“待出餐”。

查看订单与结算的用例规约如表3. 6所示。

表3. 6用例“查看订单与结算”的用例规约

用例名称：

查看订单与结算

用例描述：

普通用户查看自己的历史订单，并对满足条件的订单执行余额结算。

参与者：

普通用户

前置条件：

用户已登录系统；结算时订单状态为“待结算”，且用户余额充足。

基本操作：

用户进入订单页面并查询个人订单列表。

系统按当前用户身份返回订单编号、内容、总价和状态信息。

当订单状态变为“待结算”时，用户发起结算请求。

系统校验订单归属、订单状态和账户余额，扣减余额后将订单更新为“已完成”。

个人资料管理的用例规约如表3. 7所示。

表3. 7用例“个人资料管理”的用例规约

用例名称：

个人资料管理

用例描述：

普通用户查看和维护个人资料，包括姓名、手机号、性别、头像及余额展示。

参与者：

普通用户

前置条件：

用户已登录系统。

基本操作：

用户进入个人中心页面并查看当前资料。

系统返回用户姓名、手机号、性别、头像和余额信息。

用户修改个人资料或上传头像后提交保存请求。

系统更新用户资料，并在需要时完成头像文件上传与访问路径保存。

管理员登录的用例规约如表3.8所示。

表3.8用例“管理员登录”的用例规约

用例名称：

管理员登录

用例描述：

管理员通过 Web 管理端登录后台系统，进入业务管理页面。

参与者：

管理员

前置条件：

管理员账号已存在，且已打开后台登录页面。

基本操作：

管理员输入用户名和密码并提交登录请求。

系统校验管理员身份和角色信息。

校验通过后系统返回登录成功结果。

前端保存管理员信息并跳转到后台主界面。

管理订单的用例规约如表3.9所示。

表3.9用例“管理订单”的用例规约

用例名称：

管理订单

用例描述：

管理员对订单进行查询、状态维护和删除等后台管理操作。

参与者：

管理员

前置条件：

管理员已成功登录后台系统。

基本操作：

管理员进入订单管理页面查看订单列表。

系统分页返回订单编号、用户、金额、内容和当前状态。

管理员根据业务需要将订单状态由“待出餐”更新为“待结算”。

管理员可对无效订单执行删除操作，系统同步更新订单数据。

管理餐桌的用例规约如表3.10所示。

表3.10用例“管理餐桌”的用例规约

用例名称：

管理餐桌

用例描述：

管理员维护餐桌基础信息，并查看餐桌当前占用状态。

参与者：

管理员

前置条件：

管理员已成功登录后台系统。

基本操作：

管理员进入餐桌管理页面查询餐桌列表。

系统返回餐桌编号、规格、空闲状态和占用用户信息。

管理员根据需要新增、修改或删除餐桌记录。

系统保存变更结果，并在列表中展示最新餐桌状态。

管理菜品的用例规约如表3.11所示。

表3.11用例“管理菜品”的用例规约

用例名称：

管理菜品

用例描述：

管理员维护菜品信息，包括名称、价格、描述和图片资源。

参与者：

管理员

前置条件：

管理员已成功登录后台系统。

基本操作：

管理员进入菜品管理页面查看现有菜品列表。

管理员新增或修改菜品名称、价格、描述及图片信息。

系统完成文件上传和菜品数据保存。

管理员可删除不再提供的菜品，系统同步更新菜品列表。

管理用户的用例规约如表3.12所示。

表3.12用例“管理用户”的用例规约

用例名称：

管理用户

用例描述：

管理员查看用户信息并维护用户资料及账户余额。

参与者：

管理员

前置条件：

管理员已成功登录后台系统。

基本操作：

管理员进入用户管理或个人信息管理页面查询用户列表。

系统返回用户账号、姓名、联系方式、角色和余额信息。

管理员可修改用户资料，或根据充值需求调整账户余额。

系统保存更新结果并反馈处理状态。

管理员管理的用例规约如表3. 13所示。

表3. 13用例“管理员管理”的用例规约

用例名称：

管理员管理

用例描述：

管理员对后台管理员账号进行新增、修改、删除和查询维护。

参与者：

管理员

前置条件：

管理员已成功登录后台系统，且具备管理员维护权限。

基本操作：

管理员进入管理员管理页面查看现有管理员列表。

管理员新增管理员账号并录入基础信息。

系统默认新建账号角色为 ADMIN，并保存管理员记录。

管理员可继续修改或删除指定管理员信息。

统计订单收入的用例规约如表3. 14所示。

表3. 14用例“统计订单收入”的用例规约

用例名称：

统计订单收入

用例描述：

管理员统计已完成订单的收入数据，用于掌握系统经营情况。

参与者：

管理员

前置条件：

管理员已成功登录后台系统，且系统中存在已完成订单数据。

基本操作：

管理员进入订单管理或统计页面发起收入统计请求。

系统筛选状态为“已完成”的订单记录。

系统汇总订单总金额并返回统计结果。

页面展示收入数据，供管理员查看和分析。

3.3 功能性需求分析

系统的主要功能模块包括用户认证与登录、餐桌管理、菜品浏览与购物车、订单处理与结算、个人中心以及后台运营管理等模块。各模块之间既相对独立，又共同服务于“选座一点餐一下单一出餐一结算”的业务主线。

用户基础服务需求

用户基础服务模块主要为普通用户提供注册、登录和登录态维护功能。

餐桌与点餐业务需求

餐桌与点餐业务模块需要支持用户查看全部餐桌状态和当前餐桌信息，并完成占桌、退桌和占用冲突校验。点餐业务则需要支持用户浏览并下单。

订单交易处理需求

订单模块管理员需要记录订单编号、订单内容、订单金额和订单状态，用户支持查询个人订单并跟踪状态变化。

后台运营管理需求

后台运营管理模块主要为管理员提供订单、餐桌、菜品、充值、用户和管理员信息的维护能力。

3.4 非功能性需求分析

非功能性需求作为系统质量属性的重要体现，虽然不直接对应某一个业务操作，却会影响系统整体运行效果 [8]：

易用性方面，系统界面需要做到简洁直观，管理员端菜单层次清晰，小程序端操作流程要符合用户日常点餐习惯。

性能方面，系统需要支持常规数据的快速查询与更新，核心操作具有较好的响应速度，保证运行流畅。

安全性方面，系统需要保证关键业务校验的有效性，避免安全性等问题，同时对数据变更等敏感操作进行一定限制。

可维护性方面，系统应便于后续进行扩展、修复和维护。

通过对上述非功能性需求进行明确约束，系统不仅可以完成基本订餐业务，还可以在稳定运行和后续扩展方面形成较好的基础。

3.5 可行性分析

Spring Boot 构建在 Spring 框架基础之上，具备快速开发和生态集成较好等特点，能够较高效地完成 REST 接口、拦截器、统一异常处理和业务服务组织，适合作为本系统的后端开发框架[1]。

MyBatis 是开源、轻量级的数据持久化框架，便于开发者灵活编写查询和更新语句；结合 Mapper 接口与 XML 映射文件后，可以较好地支撑数据的持久化需求[2]。

Vue.js 是基于 JavaScript 的用户界面开发框架，其组件化开发、响应式数据绑定和路由管理机制能够提高后台管理端的开发效率；结合 Vite 与 Element Plus 组件库，可以较快构建结构清晰、交互完整的管理页面[3][4]。

微信小程序属于轻量化移动应用形态，具有无需下载安装、使用门槛低和贴近日常使用场景等优势 [5]。

综上所述，本系统采用 Java 作为主要编程语言，搭配 MySQL 5.7.44 作为数据库；前端选用 Vue 技术并结合 Element Plus 组件完成管理端界面设计，同时以微信小程序作为用户端入口。该项目在技术实现上具有较高可行性[1-6]。

4 订餐系统设计

本章主要对食堂订餐系统的总体设计进行说明。结合前文需求分析结果，系统设计从系统架构、功能设计、数据设计和前后端接口设计四个方面展开。系统以 Spring Boot 作为后端核心，利用 Vue 3 构建管理端，以微信小程序作为用户端。

4.1 系统架构设计

本系统采用前后端分离的三层架构设计。表现层由 Vue 3 Web 管理端和微信小程序用户端组成；业务层由 Spring Boot 服务承担核心逻辑；数据访问层基于 MyBatis 与 MySQL 完成数据的持久化处理 [7]。

图4.1 系统架构图

4.2 功能设计

4.2.1 功能框图

系统各功能模块的组成关系通过功能框图进行展示，系统功能框图如图4.2所示。

图4.2 系统功能框图

4.2.2 用户注册与登录功能

用户访问系统时首先需要完成身份认证。普通用户进入微信小程序登录页面后，在表单中输入账号和密码，并可根据实际情况选择注册或登录。若用户执行注册操作，系统会先对账号信息进行校验；当账号已存在或表单信息不符合要求时，页面给出相应提示，用户需要重新填写信息；当校验通过后，系统调用注册接口写入用户数据，并为该账号设置普通用户角色。若用户执行登录操作，系统会将账号和密码提交到服务器进行验证；验证成功后服务器返回用户信息和 token，小程序将这些信息保存到本地并进入业务页面；验证失败时，页面提示用户重新输入账号和密码。管理员通过 Web 管理端登录后台，系统在校验管理员账号后跳转至管理主界面。用户注册与登录流程图如图4.3。

图4.3 用户注册与登录功能流程图

4.2.3 选座与餐桌占用功能

用户完成登录后即可进入选座页面，开始餐桌占用流程。系统首先查询该用户当前是否已经占用餐桌，若已经存在占桌记录，则在页面中展示当前餐桌信息，用户不能重复选择其他餐桌；若不存在占桌记录，则继续加载全部餐桌数据，并按照空闲状态进行展示。当用户选择某个空闲餐桌并提交后，系统会对用户占桌情况和餐桌占用状态进行判断。若用户已经占用其他餐桌，或目标餐桌已经被其他用户占用，系统会提示占桌失败并要求用户重新选择；若校验通过

，则调用占桌接口，将用户信息与餐桌进行绑定，并把餐桌状态更新为占用。就餐结束后，用户可以发起退桌请求，系统解除用户与餐桌之间的绑定关系，并将餐桌恢复为空闲状态。选座与餐桌占用流程图如图4.4。

图4.4 选座与餐桌占用功能流程图

4.2.4 点餐下单功能

用户完成选座后进入点餐页面，系统首先加载当前餐桌信息和菜品列表，并展示菜品名称、价格、图片和描述等内容。用户根据需要进行菜品数量并加入购物车，系统同步计算购物车中的菜品数量和订单总金额。当用户点击提交订单时，系统先判断用户是否已经占用餐桌以及购物车中是否存在菜品；若条件不满足，则提示用户先选座或添加菜品；若条件满足，则整理菜品名称、数量、价格和总价等信息，并向服务器发送下单请求。服务器接收请求后生成订单编号，记录下单时间，绑定当前用户信息，并将订单初始状态设置为“待出餐”。若请求处理成功，页面提示下单成功并清空购物车；若请求失败，系统提示用户重新提交订单。点餐下单流程图如图4.5。

图4.5 点餐下单功能流程图

4.2.5 出餐结算与订单管理功能

订单提交成功后进入出餐和结算流程。管理员进入后台订单管理页面后，可以查看系统中的订单编号、下单用户、订单内容、总价和订单状态，并根据食堂出餐进度对订单状态进行维护。当订单仍处于“待出餐”状态时，管理员完成出餐后可将其更新为“待结算”；若订单数据异常，也可以在后台执行删除等管理操作。普通用户进入订单页面后，系统根据当前登录用户查询个人订单列表。若订单尚未出餐，用户只能查看订单状态；若订单状态已经变为“待结算”，用户可以点击结算按钮发起付款请求。系统收到结算请求后，会校验订单是否属于当前用户、订单状态是否允许结算以及账户余额是否充足；若校验不通过，页面显示失败提示；若校验通过，系统扣减用户余额，将订单状态更新为“已完成”，并把已完成订单计入后台收入统计。出餐结算与订单管理流程图如图4.6。

图4.6 出餐结算与订单管理功能流程图

4.2.6 用户资料与充值管理功能

用户资料与充值管理主要用于维护个人信息和账户余额。普通用户进入个人中心页面后，系统会查询当前用户的姓名、手机号、性别、头像和余额等信息，并将结果展示在页面中。用户可以根据需要修改个人资料，若只修改文字信息，则系统直接调用用户信息更新接口保存数据；若用户需要更换头像，则先通过文件上传接口上传图片，服务器将图片保存到指定目录并返回访问地址，随后系统再把头像地址更新到用户资料中。管理员在后台可以查看用户列表和用户余额，并根据实际充值情况修改账户金额。若保存成功，系统刷新用户信息并提示操作完成；若请求失败，则提示用户重新提交。用户资料与充值管理流程图如图4.7。

图4.7 用户资料与充值管理功能流程图

4.3 数据设计

4.3.1 概念模型（ER模型）

本系统的概念模型主要围绕管理员、用户、餐桌、菜品和订单五类核心实体展开，各实体及其属性如下：

- 1) 管理员信息属性有：id、username、password、name、avatar、role，分别表示管理员编号、登录账号、登录密码、管理员姓名、头像地址和角色标识，如图4.7所示。
- 2) 用户信息属性有：id、username、password、name、avatar、role、sex、phone、account，分别表示用户编号、登录账号、登录密码、姓名、头像、角色、性别、手机号和账户余额，如图4.8所示。

3) 餐桌信息属性有: id、no、unit、free、userId、userName, 分别表示餐桌编号、桌号、餐桌规格、空闲状态、当前占用用户编号和当前占用用户姓名, 如图4. 9所示。

4) 菜品信息属性有: id、name、descr、price、img, 分别表示菜品编号、菜品名称、菜品描述、菜品价格和菜品图片, 如图4. 10所示。

5) 订单信息属性有: id、content、total、userId、userRole、time、status、orderNo、userName, 分别表示订单编号、订单内容、订单总价、下单用户编号、下单用户角色、下单时间、订单状态、订单号和下单用户名, 如图4. 11所示。

图4. 7 管理员信息属性

图4. 8 用户信息属性

图4. 9 餐桌信息属性

图4. 10 菜品信息属性

图4. 11 订单信息属性

不同实体之间的联系主要包括以下几类:

1) 用户实体与餐桌实体之间存在“占用”联系。

用户信息: 一个用户在同一时刻最多只能占用一张餐桌, 也可以处于未占桌状态。

餐桌信息: 一张餐桌在同一时刻最多只能被一个用户占用, 也可以保持空闲状态。

2) 用户实体与订单实体之间存在“下单”联系。

订单信息: 一个用户可以提交多个订单, 但每个订单只归属于一个用户, 因此用户与订单之间构成一对多关系。

3) 订单实体与菜品实体之间存在“包含”联系。

订单信息: 一个订单可以包含多种菜品, 订单总价由所选菜品数量和价格共同决定。

菜品信息: 同一种菜品可以被多个用户重复点选, 并出现在不同订单中。

联系说明: 从概念模型来看, 订单与菜品之间属于多对多联系; 当前系统在物理实现中将订单内容以字符串形式保存在 content 字段中, 尚未拆分为独立的订单明细表。该处理方式可以简化轻量级系统的实现, 但在后续扩展统计分析、退菜或单品维度结算功能时, 仍有继续规范建模的空间。

4) 管理员实体与用户、菜品、餐桌及订单实体之间存在“维护”和“处理”联系。

联系说明: 管理员主要负责用户资料维护、菜品与餐桌基础数据管理, 以及订单状态处理和后台统计。

综合以上分析, 可以形成本系统完整的实体—联系模型。

完整实体—联系图如图4. 12所示。

图4. 12 完整实体—联系图

4.3.2 物理模型（数据字典设计）

本系统使用 MySQL 5.7.44 作为数据库，数据库可视化工具为 Navicat 17 for MySql。MySQL 作为关系型数据库能够较好地支撑订单、用户、餐桌和菜品等结构化数据的存储与查询[6]。

订单信息表

该表主要记录用户下单后生成的订单主信息，orders 表如表4.1所示。

表4.1 orders表

字段

字段类型

字段解释

id

int(11)

主键，自增ID

content

varchar(1000)

订单内容字符串

total

decimal(10,0)

订单总金额

user_id

int(11)

下单用户ID

user_role

varchar(20)

下单角色

time

varchar(255)

下单时间

status

varchar (255)

订单状态

order_no

varchar (255)

订单编号

菜品信息表

该表主要记录菜品名称、简介、价格和图片等基础信息，foods 表如表4. 2所示。

表4. 2 foods表

字段

字段类型

字段解释

id

int (11)

主键，自增ID

name

varchar (255)

菜品名称

descr

varchar (255)

菜品简介

price

decimal (10, 2)

菜品价格

img

varchar (255)

菜品图片地址

餐桌信息表

该表主要记录餐桌编号、规格、空闲状态以及当前占用用户等信息，tables 表如表4. 3所示。

表4. 3 tables表

字段

字段类型

字段解释

id

int(11)

主键，自增ID

no

varchar(255)

餐桌编号

unit

varchar(255)

餐桌规格

free

varchar(255)

餐桌是否空闲（是/否）

user_id

int(11)

当前占用用户ID

用户信息表

该表主要记录普通用户的账号、资料和余额等信息，user 表如表4. 4所示。

表4. 4 user表

字段

字段类型

字段解释

id

int(11)

主键，自增ID

username

varchar(255)

用户账号

password

varchar(255)

用户登录密码

name

varchar(255)

用户姓名

avatar

varchar(255)

用户头像地址

role

varchar(255)

用户角色标识

sex

varchar(255)

用户性别

phone

varchar(255)

用户手机号

account

decimal (10, 2)

用户账户余额

管理员信息表

该表主要记录后台管理员的基础信息，admin 表如表4. 5所示。

表4. 5 admin表

字段

字段类型

字段解释

id

int (11)

主键，自增ID

username

varchar (255)

管理员账号

password

varchar (255)

管理员登录密码

name

varchar (255)

管理员姓名

avatar

varchar (255)

管理员头像地址

role

varchar (255)

管理员角色标识

4. 4 前后端接口设计

4. 4. 1 登录模块接口

登录模块主要负责普通用户注册、身份校验和登录态建立。小程序端主要通过 /auth/register 和 /auth/login 完成账号创建与登录，管理员端则保留 /login 作为兼容入口。

该接口用于普通用户注册，详细说明如表4. 6所示：

表4. 6 用户注册接口说明

URL

/auth/register

HTTP请求方式

POST

携带参数

请求体包含 username、password、name、role；其中 role 固定为 USER，用户名需保持唯一。

回调参数

返回统一 Result 对象，包含 code、msg、data；注册成功时 data 通常为空。

功能说明

用于微信小程序端创建普通用户账号，后端完成账号唯一性校验并写入用户基础信息。

该接口用于用户登录与令牌签发，详细说明如表4. 7所示：

表4. 7 用户登录接口说明

URL

/auth/login

HTTP请求方式

POST

携带参数

请求体包含 username、password、role；普通用户 role 为 USER，管理员兼容接口可使用 /login。

回调参数

返回 Result 对象；data 中包含 id、username、name、avatar、role、token 等登录态信息。

功能说明

用于校验账号密码并签发 JWT Token，前端登录成功后将 token 保存到本地并用于后续鉴权请求。

4. 4. 2 用户信息管理模块接口

用户信息管理模块主要用于支撑“我的”页面资料展示与维护，前端通过已登录用户的 token 访问当前用户接口，不需要显式传递 userId。

该接口用于查询当前登录用户资料，详细说明如表4. 8所示：

表4. 8 当前用户信息查询接口说明

URL

/user/me

HTTP请求方式

GET

携带参数

请求头携带 Authorization: Bearer <token>; 无业务请求体。

回调参数

返回 Result 对象；data 为当前用户信息，包含 id、username、name、avatar、sex、phone、account、role 等字段。

功能说明

用于查询当前登录用户的个人资料和账户余额，为小程序个人中心页面初始化数据。

该接口用于更新当前登录用户资料，详细说明如表4. 9所示：

表4. 9 当前用户信息更新接口说明

URL

/user/me

HTTP请求方式

PUT

携带参数

请求头携带 Authorization: Bearer <token>; 请求体可包含 name、phone、sex、avatar 等需更新字段。

回调参数

返回 Result 对象；data 为更新后的最新用户信息。

功能说明

用于修改当前用户个人资料，后端根据 token 识别用户身份并完成资料更新。

4. 4. 3 餐桌和菜品模块接口

餐桌和菜品模块是用户点餐前的重要准备环节，其中餐桌接口负责占座约束控制，菜品接口负责为前端菜单页提供可视化菜品数据。

该接口用于当前用户占用餐桌，详细说明如表4. 10所示：

表4. 10 当前用户占桌接口说明

URL
/tables/current
HTTP请求方式
POST
携带参数
请求头携带 Authorization: Bearer <token>; 请求体主要包含餐桌 id，后端结合当前用户编号完成占桌。
回调参数
返回 Result 对象；data 为最新餐桌信息，包含 id、no、unit、free、userId、userName 等字段。

功能说明

用于当前用户占用指定餐桌，后端同时校验 “一个用户只能占一个餐桌、一个餐桌只能被一个用户占用” 的业务规则。

该接口用于查询菜品列表，详细说明如表4. 11所示：

表4. 11 菜品列表查询接口说明

URL
/foods/selectAll
HTTP请求方式
GET
携带参数
可选查询参数 name；小程序端通常不传该参数以获取全部菜品列表。

回调参数

返回 Result 对象；data 为菜品集合，每项包含 id、name、descr、price、img 等字段。

功能说明

用于加载菜品列表、价格、图片与描述信息，为菜单浏览和购物车组装提供数据来源。

4. 4. 4 订单结算模块接口

订单结算模块贯穿下单、订单状态流转和余额扣减等关键业务，前端既需要提交购物车生成订单，也需要在待结算状态下发起结算请求。

该接口用于创建订单，详细说明如表4. 12所示：

表4. 12 订单创建接口说明

URL

/orders

HTTP请求方式

POST

携带参数

请求头携带 Authorization: Bearer <token>; 请求体包含 content、total 等订单核心数据，userId、orderNo、time 由后端自动补全。

回调参数

返回 Result 对象；data 为创建后的订单信息，包含订单编号、下单时间、状态和所属用户等字段。

功能说明

用于将前端购物车内容提交为正式订单，后端自动生成订单编号并将订单初始状态设置为“待出餐”。

该接口用于结算订单，详细说明如表4. 13所示：

表4. 13 订单结算接口说明

URL

/orders/{id}/settle

HTTP请求方式

POST

携带参数

路径参数 `id` 为订单主键；请求头携带 `Authorization: Bearer <token>`；请求体为空。

回调参数

返回 `Result` 对象；`data` 为结算后的订单信息，订单状态更新为“已完成”。

功能说明

用于对当前用户名下处于“待结算”状态的订单执行余额扣减和状态更新，并完成订单闭环。

4.4.5 文件上传和下载模块接口

文件资源模块主要服务于用户头像和菜品图片管理，前端先上传文件并获得访问地址，再通过下载接口在页面中进行展示。

该接口用于上传图片文件，详细说明如表4.14所示：

表4.14 文件上传接口说明

URL

`/files/upload`

HTTP请求方式

POST

携带参数

表单参数 `file`，内容类型为 `multipart/form-data`；请求体携带待上传的头像或菜品图片文件。

回调参数

返回 `Result` 对象；`data` 为可直接访问的文件 URL，通常形如 `/files/download/{fileName}`。

功能说明

用于将头像或菜品图片保存到 `photo` 目录，并向前端返回后续可访问的资源地址。

该接口用于下载或访问图片资源，详细说明如表4.15所示：

表4.15 文件下载接口说明

URL

`/files/download/{fileName}`

HTTP请求方式

GET

携带参数

路径参数 `fileName` 为文件名；无需请求体。

回调参数

接口直接返回文件二进制流，响应头包含内容类型和文件名；文件不存在时返回 404。

功能说明

用于根据文件名读取服务器中的图片资源，支撑小程序头像、菜品图片和后台资源预览。

5 订餐系统实现

5.1 开发环境

处理器：AMD RYZEN7 5000 SERIES

操作系统：Windows 10

开发及测试平台：本系统在 Windows 10 环境下完成开发与联调，后端采用 IntelliJ IDEA 2024、JDK 21、Maven 3.9 和 MySQL 5.7，Web 管理端采用 Vue 3.5、Vite 6、Node.js 18 和 Element Plus，微信用户端使用微信开发者工具进行调试，接口测试和页面验证通过本地 9090 端口服务完成。

数据库：MySQL 5.7.44

数据库可视化工具：Navicat 17 for MySQL

5.2 系统关键代码

本节选取系统中最能体现食堂订餐业务规则的两组关键代码进行说明：一组是餐桌占用与退桌控制代码，另一组是订单创建、结算与余额扣减代码。登录鉴权、请求封装等内容主要属于系统基础支撑，因此不再作为本节重点展开。

5.2.1 餐桌占用与状态控制关键代码

餐桌占用逻辑是用户点餐前的前置环节，直接决定食堂座位资源能否被正确分配。该模块通过当前登录用户编号和餐桌编号建立绑定关系，并在服务层校验“一个用户同一时间只能占用一个餐桌、一个餐桌同一时间只能被一个用户占用”的核心规则。

用户占桌成功后，系统将餐桌状态由“是”更新为“否”，并写入占用用户编号；当用户退桌时，再根据当前用户编号查找已占用餐桌并释放绑定关系，从而保证餐桌状态与用户操作保持一致。关键代码如下：

餐桌占用与退桌服务代码：

```
@PostMapping("/current")

public Result reserveCurrent(@RequestBody Tables tables) {

    Tables latestTable = tablesService.reserveCurrentTable(

        tables.getId(), AuthContext.getCurrentUserId());

    return Result.success(latestTable);
}
```



```

}

@DeleteMapping("/current")

public Result removeCurrent() {

    tablesService.removeCurrentTable(AuthContext.getCurrentUserId());

    return Result.success();

}

public Tables reserveCurrentTable(Integer tableId, Integer currentUserId) {

    if (currentUserId == null) {

        throw new CustomException("请先登录");

    }

    if (tableId == null) {

        throw new CustomException("请选择餐桌");

    }

    Tables tables = new Tables();

    tables.setId(tableId);

    tables.setUserId(currentUserId);

    this.addOrder(tables);

    return this.selectById(tableId);

}

public void addOrder(Tables tables) {

    Tables dbTables = tablesMapper.selectByUserId(tables.getUserId());

    if (dbTables != null && !dbTables.getId().equals(tables.getId())) {

        throw new CustomException("您已经预定了其他餐桌");

    }

    Tables currentTable = tablesMapper.selectById(tables.getId());

    if (currentTable == null) {

```

```

throw new CustomException("餐桌不存在");
}

if (!"是".equals(currentTable.getFree())

&& !Objects.equals(currentTable.getUserId(), tables.getUserId())) {

throw new CustomException("该餐桌已被占用");

}

tables.setFree("否");

this.updateById(tables);

}

public void removeCurrentTable(Integer currentUserId) {

if (currentUserId == null) {

throw new CustomException("请先登录");

}

Tables tables = tablesMapper.selectByUserId(currentUserId);

if (tables == null) {

return;

}

tablesMapper.removeOrder(tables.getId());

}

```

5.2.2 订单创建与结算关键代码

订单创建与结算逻辑是本系统业务闭环的核心。用户提交购物车后，后端不直接信任客户端传入的用户编号，而是从登录上下文中取得当前用户，将订单绑定到本人名下，并自动生成订单编号和下单时间。

结算时，系统会校验订单是否存在、是否属于当前用户以及是否处于“待结算”状态。校验通过后再扣减用户余额，并将订单状态更新为“已完成”，从而实现从下单到支付完成的完整流程。关键代码如下：

订单创建、结算与余额扣减服务代码：

```

@PostMapping

public Result create(@RequestBody Orders orders) {

```

```

Orders latestOrder = ordersService.addForCurrentUser(
orders, AuthContext.getCurrentUserId());

return Result.success(latestOrder);
}

@PostMapping("/{id}/settle")

public Result settle(@PathVariable Integer id) {

Orders orders = ordersService.settleOrder(

id, AuthContext.getCurrentUserId());

return Result.success(orders);
}

public Orders addForCurrentUser(Orders orders, Integer currentUserId) {

if (currentUserId == null) {

throw new CustomException("请先登录");
}

orders.setUserId(currentUserId);

this.add(orders);

return orders;
}

public void add(Orders orders) {

this.prepareOrderForInsert(orders);

String orderNo = IdUtil.fastSimpleUUID();

orders.setOrderNo(orderNo);

orders.setTime(DateUtil.now());

ordersMapper.insert(orders);
}

public Orders settleOrder(Integer orderId, Integer currentUserId) {

```

```
if (currentUserId == null) {

throw new CustomException("请先登录");

}

Orders dbOrder = this.selectById(orderId);

if (dbOrder == null) {

throw new CustomException("订单不存在");

}

if (!Objects.equals(dbOrder.getUserId(), currentUserId)) {

throw new CustomException("无权结算该订单");

}

if ("已完成".equals(dbOrder.getStatus())) {

return dbOrder;

}

if (!"待结算".equals(dbOrder.getStatus())) {

throw new CustomException("当前订单状态不允许结算");

}

this.deductUserBalance(dbOrder);

Orders updateOrder = new Orders();

updateOrder.setId(orderId);

updateOrder.setStatus("已完成");

ordersMapper.updateById(updateOrder);

dbOrder.setStatus("已完成");

return dbOrder;

}

private void deductUserBalance(Orders orders) {

User user = userMapper.selectById(orders.getUserId());
```

```
if (user == null) {  
  
throw new CustomException("用户不存在");  
  
}  
  
BigDecimal total = orders.getTotal();  
  
BigDecimal account = user.getAccount().subtract(total);  
  
if (account.doubleValue() < 0) {  
  
throw new CustomException("账户余额不足");  
  
}  
  
user.setAccount(account);  
  
userMapper.updateById(user);  
  
}
```

6 系统测试

系统测试采用黑盒测试方法，并结合本课题“Spring Boot 后端 + Vue 管理端 + 微信小程序用户端”的实现方式，重点对菜品管理、用户注册与登录、餐桌占用与退桌、订单创建与结算等关键业务功能进行验证。测试结果表明，各模块能够按照预期完成数据交互和状态流转，具体测试情况如表6.1~表6.4所示：

表6.1 菜品管理功能测试

测试编号：Test01

测试项目：菜品管理的基本操作（增删改查）。

测试目的：验证管理员对菜品信息的新增、查询、修改和删除功能是否正常，确保菜品名称、价格、描述与图片等数据能够正确保存和回显。

测试过程：1. 管理员登录 Web 管理端，进入“菜品管理”页面。2. 点击“新增”按钮，录入菜品名称、价格、描述并上传图片后保存。3. 在列表中查询新增菜品，确认页面已成功显示测试数据。4. 点击“编辑”修改菜品价格或描述后再次保存。5. 点击“删除”移除测试数据并刷新列表检查结果。

预期结果：新增后列表能够立即显示新菜品；修改后页面展示最新信息；删除后记录从列表中移除；整个操作过程均返回成功提示，且不会影响其他菜品数据。

测试结果：测试通过。 **结果分析：**菜品管理模块的增删改查流程完整，前后端交互与数据持久化结果正常，能够满足后台基础数据维护需求。

图6.1 菜品管理测试结果图（建议截图管理员在“菜品管理”页面完成新增或编辑后，列表成功显示菜品名称、价格、描述和图片的界面）

表6.2 用户注册与登录功能测试

测试编号：Test02

测试项目：微信小程序端用户注册与登录流程。

测试目的：验证普通用户账号注册、登录鉴权以及登录态保存是否正常，确保用户能够顺利进入系统并访问后续业务页面。

测试过程：1. 在微信小程序登录页切换到注册模式，输入用户名、姓名、密码和确认密码。2. 点击“注册”完成账号创建。3. 切回登录模式，使用刚注册的账号和密码发起登录请求。4. 登录成功后检查页面是否自动跳转到“选座”页。5. 再进入“我的”或其他业务页，确认无需重复登录。

预期结果：注册成功后系统返回成功提示并回到登录模式；登录成功后自动跳转至系统主页；本地登录态生效，用户可以继续访问选座、点餐和个人资料等页面。

测试结果：测试通过。 结果分析：注册与登录流程可以正常建立用户会话，用户侧鉴权功能有效，为后续餐桌、订单和个人信息操作提供了可靠的身份基础。

图6.2 用户注册与登录测试结果图（建议截图小程序使用新注册账号登录成功后自动跳转到“选座”页的界面，体现登录态已建立）

表6.3 餐桌占用与退桌功能测试

测试编号：Test03

测试项目：用户选座、占桌与退桌过程测试。

测试目的：验证“一个用户同一时间只能占用一个餐桌、一个餐桌同一时间只能被一个用户占用”的业务规则是否正确执行，并检查退桌后的状态恢复情况。

测试过程：1. 用户登录小程序进入“选座”页面，查看当前餐桌列表与空闲状态。2. 选择一个空闲餐桌并提交占桌请求。3. 进入点餐页或返回选座页，检查当前餐桌信息是否正确显示。4. 使用同一用户再次尝试占用其他餐桌，观察系统提示信息。5. 执行退桌操作后刷新页面，检查原餐桌是否恢复为空闲状态。

预期结果：首次占桌成功后页面显示当前餐桌信息；再次占用其他餐桌时系统能够拦截并提示；退桌后当前餐桌信息被清空，原餐桌重新显示为空闲状态。

测试结果：测试通过。 结果分析：餐桌绑定与释放逻辑运行正确，能够有效避免餐桌冲突，符合食堂场景下的实际座位管理要求。

图6.3 餐桌占用测试结果图（建议截图用户成功占用餐桌后页面显示“当前餐桌编号”和已进入点餐状态的界面，或退桌后餐桌重新为空闲的列表界面）

表6.4 订单创建与结算功能测试

测试编号：Test04

测试项目：订单创建、状态流转与余额结算测试。

测试目的：验证用户从点餐下单、管理员处理订单到用户完成结算的完整业务链路，并检查订单状态变更与余额扣减结果是否准确。

测试过程：1. 用户先完成选座，在菜单页选择菜品加入购物车并提交订单。2. 进入“我的订单”页，检查新订单是否生成，初始状态是否为“待出餐”。3. 管理员进入 Web 管理端订单页面，将该订单状态更新为“待结算”。4. 用户返回小程序订单页点击“结算”，确认扣款金额后提交。5. 结算完成后检查订单状态是否变为“已完成”，并核对用户余额是否按订单总额减少。

预期结果：订单能够成功创建；状态按照“待出餐→待结算→已完成”的顺序流转；余额扣减金额与订单总额一致；后台收入统计仅对已完成订单生效。

测试结果：测试通过。结果分析：订单模块已实现从下单到结算的业务闭环，核心状态控制、余额扣减和后台收入统计逻辑均符合系统设计要求。

图6.4 订单结算测试结果图（建议截图小程序“我的订单”页面中订单状态变为“已完成”的界面，或后台订单管理页面将订单更新为“待结算”的界面）

结 论

本文围绕订餐业务，设计并实现了一套基于 Web 的订餐系统。系统采用前后端分离架构，后端以 Spring Boot 作为核心服务框架，结合 MyBatis 和 MySQL 完成业务数据持久化；Web 管理端采用 Vue 3 与 Element Plus 构建管理员界面，微信小程序端则给普通用户点餐。

系统实现和测试结果表明，用户注册与登录、餐桌占用与退桌、订单创建与结算三个重点流程均能够按照预期完成。系统能够正确建立用户登录态，能够对餐桌占用冲突进行校验，也能够对订单进入待结算状态后完成余额扣减和订单状态更新，说明系统的主要业务逻辑具备可用性。

系统仍然有不足之处，当前管理员端鉴权机制仍有进一步完善空间，用户与管理员密码尚未进行加密存储，订单内容仍以字符串形式保存，尚未拆分为独立订单明细表；充值与扣款也缺少独立资金流水记录。虽然不影响正常使用，但会有一定缺失性。

参 考 文 献

- [1] VMware. Spring Boot Reference Documentation[EB/OL]. <https://docs.spring.io/spring-boot/index.html>, 2026-05-19.
- [2] MyBatis Team. MyBatis 3 User Guide[EB/OL]. <https://mybatis.org/mybatis-3/index.html>, 2026-05-19.
- [3] Vue.js Team. Vue.js Guide[EB/OL]. <https://vuejs.org/guide/>, 2026-05-19.
- [4] Element Plus Team. Element Plus Documentation[EB/OL]. <https://element-plus.org/en-US/>, 2026-05-19.
- [5] Tencent. 微信小程序开发文档[EB/OL]. <https://developers.weixin.qq.com/miniprogram/dev/framework/>, 2026-05-19.
- [6] Oracle. MySQL 5.7 Reference Manual[EB/OL]. <https://dev.mysql.com/doc/refman/5.7/en/>, 2026-05-19.
- [7] Fielding R T. Architectural Styles and the Design of Network-based Software Architectures[D]. Irvine: University of California, 2000.

[8] ISO/IEC. ISO/IEC 25010:2011 Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models[S]. Geneva: ISO, 2011.

[9] Myers G J, Sandler C, Badgett T. The Art of Software Testing[M]. 3rd ed. Hoboken: John Wiley & Sons, 2011.

[10] Sommerville I. Software Engineering[M]. 10th ed. Boston: Pearson, 2016.

致 谢

大学生活已近尾声，非常感谢在大学生活中帮助和鼓励过我的人。

首先，我非常感谢我的指导老师张玲老师。在我不知道怎么完成毕业设计的过程里，她的专业知识给了我很好的指导和支持，给予了我无比宝贵的经验，我顺利完成毕业设计和论文离不开她的帮助

最后，我要感谢我父母，没有他们的支持，我很难走到今天。